

ECOM EXPERTS

Product Fields manager 8.2

Manual de usuario e instalacion

an_productfields82 2.5.0 - copia 'modulos transformar' (al limite) · 04/09/2026

Contenido

1. Qué hace y para quién

Aviso: hay DOS copias de este módulo

2. Compatibilidad

3. Instalación

4. Configuración

4.1 El listado de campos

4.2 Los ajustes

4.3 Crear un campo

5. Cómo se usa, día a día

6. Qué ve el cliente en la tienda

7. Preguntas frecuentes y problemas

8. Seguridad

Corregido

Lo que ya estaba bien

Pendiente (no se ha tocado a propósito)

9. Ficha técnica

Versión 2.5.0 · Revisado e instalado sobre PrestaShop 8.2.7

1. Qué hace y para quién

Este módulo añade a la ficha de producto de la tienda **campos que rellena el cliente antes de comprar**: el texto que quiere grabar en una pulsera, la fecha en la que necesita el ramo, el color de hilo entre una lista cerrada, el archivo de referencia que le pide el taller. Sin el módulo, esa información hay que pedirla por correo después de la venta, con el pedido ya cobrado y el cliente esperando; con él, llega dentro del propio pedido, pegada a la línea del producto correspondiente.

Cada campo se define una sola vez desde el back-office y se decide dónde se aplica: a **todos** los productos, a los de **unas categorías** concretas, o **producto a producto** desde la propia ficha. También se decide **quién lo ve**, porque cada campo se asigna a grupos de clientes: se puede tener un campo que solo aparezca para el grupo mayorista y no para el visitante anónimo.

Un campo puede además **llevar precio**. Si grabar el nombre cuesta 3,50 €, se pone ese importe en el campo y la ficha del producto lo enseña junto a la etiqueta («Texto grabado (+4,24 €)», con el IVA ya aplicado si así se configura) y actualiza el precio total en pantalla según el cliente va eligiendo. Ese sobreprecio se suma al carrito y viaja hasta el pedido, la factura y los correos de confirmación.

Está pensado para tiendas de **producto personalizado**: grabados, regalos, ropa a medida, material impreso, alimentación con fecha de entrega, cualquier catálogo en el que la misma referencia se venda con datos distintos según el comprador. Esta copia (`an_productfields82`) es la adaptación a PrestaShop 8.2 del módulo original `an_productfields`, y **comparte con él las mismas tablas y la misma configuración**: se puede desactivar uno y activar el otro sin perder ni un dato.

Aviso: hay DOS copias de este módulo

En el disco conviven dos carpetas con el mismo módulo, y **no son iguales**. Este manual documenta la de `instalaciones prestashop y updates / copias para instalar / al limite / modulos transformar`, que es la revisada aquí sobre PrestaShop **8.2**.

La otra, en `modulos IA / modulos convertidos a 8.2`, va **por delante**: la lleva otro trabajo en paralelo sobre PrestaShop 9.2 y no se ha tocado. Diferencias reales (`diff -r` sobre las dos carpetas: 10 ficheros distintos):

	Esta copia (8.2)	La de módulos convertidos a 8.2
Override del carrito	<code>AN_PRODUCTFIELDS82_OVERRIDE = '2.5.0' ,</code> <code>getProducts()</code> con la firma de PrestaShop 8	2.5.2 , firma adaptada a PrestaShop 9 y propiedades del carrito declaradas
Hook de la ficha	<code>displayProductButtons</code>	<code>displayProductAdditionalInfo</code> (el anterior quedó obsoleto en PS 9) y baja del viejo al actualizar
Overrides antiguos	se dejan si ya llevan la marca	además se comparan con el del módulo y se reponen si difieren
Precios en plantilla	<code>Tools::displayPrice()</code> dentro del <code>.tpl</code>	precio ya formateado en PHP (<code>anPfDisplayPrice , price_formatted</code>)
Escapes	<code>{ \$value.file }</code> escapado aquí; <code>escape: 'javascript'</code> no está	<code>escape: 'javascript'</code> en <code>script.tpl</code> , <code>rel="noopener"</code> en el enlace del listado; <code>{ \$value.file }</code> sin escapar

Las dos coinciden en las declaraciones de propiedad para PHP 8.2 y en el `.htaccess` de `views/img/` , a los que se ha llegado por separado.

Cuál es la buena: para una tienda **8.2**, esta. Para **PrestaShop 9**, la otra, que es la única con la firma de `getProducts()` y el hook correctos para esa versión. Los arreglos de seguridad de la sección 8 —la subida de ficheros, el acceso a pedidos ajenos y los endpoints por hash— **solo están en esta copia** y habría que llevarlos también a la otra.

2. Compatibilidad

PrestaShop	Probado e instalado en 8.2.7 . El código conserva las ramas de 1.6 y 1.7 (<code>\$this->new</code> , <code>\$this->morenew</code>), pero la versión que se entrega y se ha verificado es la 8.2.
PHP	<code>php -l</code> limpio en 7.4 y 8.2 sobre los 22 ficheros PHP del módulo. En 8.2 ya no salta ningún aviso de propiedad dinámica (ver sección 8).
Multitienda	Sí. Cada campo se asocia a tiendas concretas (tabla <code>an_productfields_shop</code>) y el listado del back-office filtra por el contexto de tienda. Los ajustes generales de la pestaña «Settings» son por tienda (<code>Configuration::updateValue</code>), salvo los avisos internos del override, que son globales. Si se está en contexto «Todas las tiendas», la pestaña del módulo en la ficha de producto avisa con «Please select a shop!» en lugar de dejar guardar en falso.
Idiomas	Los nombres de los campos y los valores de las listas son multiidioma (<code>an_productfields_lang</code> , columna <code>id_lang</code> en <code>an_productfields_values</code>). Los textos de cabecera y pie del bloque también. El módulo usa el sistema de traducciones clásico (<code>translations/es.php + {1 s='...' mod='an_productfields82'}</code>); la pantalla del back-office se ve en inglés salvo lo que traduce PrestaShop.
Dependencias	Ninguna externa. Sin Composer, sin <code>vendor/</code> , sin llamadas a servicios de terceros. SweetAlert va incluido en el propio módulo.
Convivencia	Puede estar instalado a la vez que <code>an_productfields</code> , porque los overrides se copian a mano y no por el instalador de PrestaShop.

3. Instalación

1. Subir la carpeta `an_productfields82` a `modules/` de la tienda (o instalar el ZIP desde **Módulos** → **Subir un módulo**).
2. Pulsar **Instalar**.
3. Entrar en **Catálogo** → **AN Product Fields**. La primera visita ejecuta las migraciones pendientes (`addnewfeatures()`) y coloca el override del carrito.

Qué crea al instalar

- **Siete tablas** (todas con `CREATE TABLE IF NOT EXISTS` , así que si ya existen del módulo original no se toca nada): `an_productfields` , `an_productfields_lang` , `an_productfields_values` , `an_productfields_shop` , `an_productfields_product` , `an_productfields_cart` , `an_productfields_cart_values` .
- **Una pestaña de back-office**: `AdminAnProductFields82` , colgando de *Catálogo*.
- **Dieciséis hooks** (lista completa en la sección 9).
- **Los valores de configuración por defecto**, pero solo los que aún no existan: si la tienda ya venía del módulo original, su configuración se respeta tal cual.

- **Un override:** `override/classes/Cart.php`, copiado a mano desde `override_v8/`, no con `installOverrides()`. Ese fichero es lo que hace que el precio del campo se sume al carrito y al total del pedido.

Qué pasa al desinstalar

- Se retira **únicamente** el override si lleva la marca del módulo; si había uno anterior, se repone desde la copia `Cart.php.an_pf_backup`.
- Se borra la pestaña del back-office.
- **Las tablas NO se borran**, porque son las mismas que usa `an_productfields` y perderlas se llevaría por delante los datos del otro módulo. Solo desaparecen si se pide expresamente el reseteo con borrado de tablas.

Al desactivar (no desinstalar) también se quita el override, para que «desactivado» signifique de verdad que el módulo deja de tocar el carrito. Al reactivarlo se repone solo.

4. Configuración

Toda la pantalla del módulo está en **Catálogo → AN Product Fields**, con un menú lateral de cuatro entradas: *Product fields* (el listado), *Settings* (los ajustes), y dos enlaces externos al soporte del autor.

4.1 El listado de campos

8.2.7 Acceso rápido 🔍 Buscar Modo depuración Ver mi tienda

Catálogo / AN Product Fields

AN Product Fields

Ayuda

⚠ El override del carrito (override/classes/Cart.php) NO está instalado: los campos no se sumarán al carrito ni al pedido. Revisa los permisos de escritura.

! El fichero C:\dev-tools\testshop\ps82b\override\classes\Cat.php ya existe y no pertenece a este modulo. No se ha tocado: el módulo no funcionará en el carrito hasta que se combine a mano.

Product fields

Settings

Support

Another modules

Rating:
If you like this module, Please RATE me 5 stars [here](#).
Thank you very much!

ID	Internal name	Price	Type	Sort Order	Date Add	Date Last Update	Enabled	
1	texto_grabado	3,50 €	text	1	2026-09-04 01:25:23	2026-09-04 01:25:23	✓	Modificar

Aquí están todos los campos creados, con su precio, su tipo y su orden. Se crean con el **+** de la esquina y se editan con **Modificar**. La columna *Enabled* apaga un campo sin borrarlo: deja de salir en la tienda pero conserva lo que ya se vendió con él.

En la captura se ven además **dos avisos** que el módulo pinta cuando el override del carrito no está puesto. No son un error del módulo: son su autocomprobación de arranque avisando de que otro módulo ya había dejado su propio `override/classes/Cart.php` en la tienda. El módulo **no lo pisa** a propósito —machacarlo rompería el otro módulo— y avisa en rojo de que, hasta que los dos overrides se combinen a mano, los precios de los campos se verán en la ficha pero **no se sumarán al carrito**. En una tienda sin ese conflicto los avisos no aparecen y el override se instala solo.

4.2 Los ajustes

 PrestaShop 8.2.7

Acceso rápido

Buscar

Modo depuración

Ver mi tienda

Ayuda

Inicio

VENDER

Pedidos

Catalogo

Productos

Categorías

Monitoreo

Atributos y Características

Marcas y Proveedores

Archivos

Descuentos

Stock

Ebooks Narcea

Mega Products

Venta por medidas

Ofertas de unidades extra

Unidades extra: consentimientos

Miele Omnigestor

AN Product Fields

Clientes

Servicio al Cliente

Estadísticas

Puntos de fidelidad

Presupuestos

PERSONALIZAR

Módulos

Diseño

Transporte

Pago

Internacional

CONFIGURAR

Catalogo / AN Product Fields

AN Product Fields

El override del carrito (override/classes/Cart.php) NO está instalado: los campos no se sumarán al carrito ni al pedido. Revisa los permisos de escritura.

El fichero C:\dev-tools\testshop\ps82b\override/classes/Cart.php ya existe y no pertenece a este modulo. No se ha tocado: el módulo no funcionará en el carrito hasta que se combine a mano.

Product fields

Settings

Support

Another modules

Rating:
If you like this module, Please RATE me 5 stars [here](#).
Thank you very much!

Expert Settings

Enable display in a separate table

Enabled

Apply tax to fields value

Enabled

Apply specific price to fields value

Enabled

Productfields header text

Personaliza tu producto

Leave this field empty if you don't want the block with productfields to have a header.

Productfields footer text

Los campos marcados con * son obligatorios.

Leave this field empty if you don't want the block with productfields to have a footer.

Product added to cart modal

Original

Simple

None

This option does not work if your theme override's "ps_shoppingcart/modal.tpl"

Dynamic product price change for storefront

Enabled

Display fields prices for storefront

Enabled

Display symbol counter for text fields

Enabled

HTML tag for price

The option is used in case when your theme was changed a lot. It's recommended not to use it without instructions of the developer.

HTML tag for product fields

product-add-to-cart

The option is used in case when your theme was changed a lot. It's recommended not to use it without instructions of the developer.

JS position for product fields

Before

After

Prepend

Append

The option is used in case when your theme was changed a lot. It's recommended not to use it without instructions of the developer.

Custom validator

ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz0123456789 .,-

Enter the characters to be successfully validated.

Corregir el precio en los correos del pedido

Enabled

Hace que el correo de confirmación muestre el precio con los campos sumados. En PrestaShop 8.2 ya no hace falta sobrescribir PaymentModule: se resuelve con el hook actionPaymentModuleProductVarTplAfter.

Clear Prestashop Cache

Disabled

Soft reset module

Disabled

If a different theme is selected, you must restart the module with the same parameters

Save

Ecom Experts

Pagina 8 de 21

Opción	Qué hace de verdad y qué poner
Enable display in a separate table	Con esto activado, los campos rellenos se imprimen en un bloque aparte en la factura PDF y se añaden a los correos <code>order_conf</code> y <code>new_order</code> . Actívalo si el taller trabaja con la factura en la mano; apágalo si prefieres que solo aparezcan en el detalle del pedido.
Apply tax to fields value	Decide si el sobreprecio del campo se enseña con IVA. Actívalo en tiendas B2C (el cliente espera ver el precio final) y apágalo en B2B. Con él activo, «3,50 €» de campo se muestra como «+4,24 €» con IVA del 21 %.
Apply specific price to fields value	Si el producto tiene un descuento porcentual, aplica ese mismo descuento al precio del campo. Actívalo si consideras el campo parte del producto; apágalo si el grabado se cobra siempre igual, esté el producto rebajado o no.
Productfields header text	Título del bloque en la ficha. Es multiidioma. Déjalo vacío si no quieres cabecera. En la captura está puesto «Personaliza tu producto».
Productfields footer text	Texto pequeño bajo los campos. Sirve para la letra pequeña: «Los campos marcados con * son obligatorios», «Revisa la ortografía, el grabado no admite cambios». Vacío = sin pie.
Product added to cart modal	Sustituye la ventana «Producto añadido al carrito» del tema <i>classic</i> . <i>Original</i> la deja como está, <i>Simple</i> pone una versión reducida y <i>None</i> la quita. Ojo: reescribe <code>themes/classic/modules/ps_shoppingcart/modal.tpl</code> (guardando antes una copia <code>modal-original.tpl</code>), así que no funciona si tu tema ya sobrescribe esa plantilla. Con un tema propio, déjalo en <i>Original</i> .
Dynamic product price change for storefront	El precio grande de la ficha se recalcula en el navegador según el cliente marca opciones con precio. Actívalo salvo que tu tema pinte el precio de una forma rara y el número salga descolocado.
Display fields prices for storefront	Enseña u oculta el «(+4,24 €)» junto a cada opción. Apágalo si prefieres que el sobrecoste aparezca solo al llegar al carrito.
Display symbol counter for text fields	Muestra el contador «(12/50)» bajo los campos de texto. Solo tiene sentido si has puesto un <i>Max Text Length</i> al campo; si el campo no tiene límite, ahora el contador no aparece (antes salía «(0/-1)», ver sección 8).
HTML tag for price / HTML tag for product fields / JS position	Selectores CSS para colocar el bloque en temas muy modificados. El valor bueno para el tema <i>classic</i> de PrestaShop 8 es <code>.product-add-to-cart</code> con posición <i>Before</i> , que es como está en la captura. No los toques sin necesidad: si el selector no existe en el tema, el bloque desaparece de la ficha.
Custom validator	Lista de caracteres permitidos. Se escriben seguidos, sin separadores: los que estén en esa cadena se aceptan y cualquier otro rechaza el formulario. Sirve para grabados donde la máquina solo admite ciertos símbolos. Luego hay que elegir <i>Personalizar</i> en la validación del campo para que se use.
Corregir el precio en los correos del pedido	Hace que el correo de confirmación muestre el precio con los campos ya sumados. El módulo original sobrescribía <code>PaymentModule::validateOrder()</code> entero para esto; en 8.2 se resuelve

Opción	Qué hace de verdad y qué poner
	con el hook <code>actionPaymentModuleProductVarTplAfter</code> y no hace falta ningún override . Déjalo activado.
Clear Prestashop Cache	No es un ajuste que se quede guardado: al pulsar <i>Save</i> con esto activado se vacía la caché de Smarty, la de XML y la de Media, y vuelve solo a <i>Disabled</i> .
Soft reset module	Igual: desinstala y reinstala el módulo sin borrar las tablas . Se usa al cambiar de tema, para que los hooks se vuelvan a registrar. Los datos de los campos no se pierden; la cabecera y el pie sí vuelven a su valor por defecto , así que anótalos antes.

Se guarda todo con el botón **Save** del pie. Comprobado contra la base de datos: los catorce valores llegan a `ps_configuration` / `ps_configuration_lang` con lo que se puso en pantalla.

PrestaShop8.2.7

Acceso rápido

Buscar

Modo depuración

Ver mi tienda

Inicio

VENDER

Pedidos

Catalogo

Productos

Categorías

Monitoreo

Atributos y Características

Marcas y Proveedores

Archivos

Descuentos

Stock

Ebooks Narcea

Mega Products

Venta por medidas

Ofertas de unidades extra

Unidades extra: consentimientos

Miele Omnigestor

AN Product Fields

Cientes

Servicio al Cliente

Estadísticas

Puntos de fidelidad

Presupuestos

PERSONALIZAR

Modulos

Diseño

Transporte

Pago

Internacional

CONFIGURAR

Catalago / AN Product Fields

Editar: Texto grabado

Ayuda

Product Attribute

* Nombre

Texto grabado

For proper displaying of fields do not use the following symbols in the name: ";", "<", ">";

* Internal name

texto_grabado

Latin letters, digits, underscores ("_") hyphens ("-") are allowed.

Price

3.50

Status

Enabled

Sort Order

1

Required

No

Type

Text

Textarea

Fecha

Radiobutton

Multiselect

Dropdown

Checkbox

Validation

None

Alphanumeric

Numeric

Alphabetic

URL

Direccion de correo electronico

Personalizar

Placeholder

Max Text Length

0

Leave the field empty if text length is required to be unlimited

* Customer Groups

ID	Nombre del grupo
☒	1 Visitante
☒	2 Invitado
☒	3 Cliente

Apply this field for all products

Enabled

Apply to all products in selected categories

Contraer todo

Expandir todo

Marcar todas

Desmarcar todas

☐ Inicio

Custom tax

None

Cancelar

Save

Casilla	Qué poner
Nombre	Lo que ve el cliente. Es multiidioma. No uses ; : , - : el módulo guarda el valor y su precio en una misma cadena separada por esos símbolos, y un nombre que los lleve rompe la línea del pedido.
Internal name	El identificador interno, solo letras, dígitos, _ y -. Es el que aparece en el listado y el que no cambia al traducir.
Price	El sobreprecio, sin impuestos . 0 para un campo que no cobra nada.
Status	Activo o no.
Sort Order	Orden en el que salen los campos dentro del bloque. Números pequeños primero.
Required	Si está activo, el cliente no puede añadir al carrito sin rellenarlo. El aviso salta en la propia ficha, antes de enviar.
Type	<i>Text</i> (una línea), <i>Textarea</i> (varias), <i>Fecha</i> (con calendario), <i>Radiobutton</i> , <i>Multiselect</i> , <i>Dropdown</i> y <i>Checkbox</i> . Los cuatro últimos exigen rellenar además una lista de valores , y cada valor puede llevar su propio precio y hasta una imagen.
Validation	<i>None</i> , <i>Alphanumeric</i> , <i>Numeric</i> , <i>Alphabetic</i> , <i>URL</i> , <i>Dirección de correo electrónico</i> y <i>Personalizar</i> (usa la lista de caracteres de los ajustes). Se comprueba en el navegador y otra vez en el servidor.
Placeholder	El texto gris de dentro de la caja.
Max Text Length	Longitud máxima. 0 o vacío = sin límite , y en ese caso el contador de caracteres no se pinta.
Customer Groups	Obligatorio marcar al menos uno. El campo solo se enseña a los grupos marcados. Para que lo vea todo el mundo, incluido quien no ha entrado, marca también <i>Visitante</i> .
Apply this field for all products	Con esto activo el campo sale en todo el catálogo y no hace falta tocar producto por producto.
Apply to all products in selected categories	Alternativa a lo anterior: el campo sale en los productos de las categorías marcadas.
Custom tax	Regla de impuestos propia para el sobreprecio, por si el grabado tributa distinto del producto. <i>None</i> usa la del producto.

Si el campo es de lista, después de guardarlo aparece la zona de **valores**: se añaden uno a uno, con su idioma, su precio y, si hace falta, una imagen de muestra. Hasta que el campo no está guardado, esa zona avisa de que primero hay que guardarlo (no es un fallo).

5. Cómo se usa, día a día

Caso 1 — Un grabado para todo el catálogo. Crear un campo *Text* llamado «Texto grabado», precio 3,50, *Apply this field for all products* activado, todos los grupos marcados y *Required* apagado. Guardar. Ya está en todas las fichas; no hay que tocar ni un producto.

Caso 2 — Un campo solo para unos productos. Crear el campo con *Apply for all products* apagado y sin categorías, guardar, y luego ir a **Catálogo** → **Productos**, abrir el producto y activar el campo en la pestaña del módulo. Ahí también se decide el orden en el que aparece en esa ficha concreta.

Caso 3 — Una lista de opciones con precios distintos. Crear el campo como *Dropdown*, guardarlo, y añadir los valores: «Sin caja» a 0, «Caja de regalo» a 4, «Caja de madera» a 12. La ficha enseña el desplegable con el sobrepeso al lado de cada opción y el precio total del producto se mueve al elegir.

Caso 4 — Ver lo que pidió el cliente. En **Pedidos**, abrir el pedido: aparece la pestaña *Product Fields* con lo que se rellenó, agrupado por producto y con la cantidad. Si además está activado *Enable display in a separate table*, esos mismos datos salen en la factura PDF y en los correos de confirmación.

Cuándo NO hay que tocar nada. Los campos ya vendidos se guardan copiados en las tablas del carrito y del pedido: cambiar el precio de un campo, renombrarlo o desactivarlo **no altera los pedidos antiguos**.

6. Qué ve el cliente en la tienda

Contacte con nosotros

Iniciar sesión

Carrito (0)

my store

CLOTHESACCESORIOSART

Búsqueda en catálogo

Inicio / Clothes / Men / Hummingbird printed t-shirt

-20%

NUEVO





HUMMINGBIRD PRINTED T-SHIRT

28,92€

23,14€20% DE DESCUENTO

Impuestos incluidos

Regular fit, round neckline, short sleeves. Made of extra long staple pima cotton.

Tamaño: S

S

Color: Blanco

☐

☒

Personaliza tu producto

Texto grabado (+4,24 €)

Los campos marcados con * son obligatorios.

Cantidad

1

AÑADIR AL CARRITO

Compartir

Este producto no admite devolución

Este producto no admite devolución ni desistimiento por tratarse de un artículo personalizado o excluido por ley (art. 103 TRLGDCU). Revise bien su elección antes de comprar.

Este artículo se fabrica en lotes de 8 unidades

Puedes llevarte 7 unidades más con un 50% de descuento

Este artículo se fabrica de 8 en 8 unidades. Como has pedido 1, en el taller sobrarían 7 unidades que acabarían en la basura.

SI ACEPTAS:

	AHORA	SI ACEPTAS
Unidades que recibes	1	8 (+7)
Precio del artículo	23,14 €	104,11 € (+80,97 €)
Te sale a	23,14 € / ud	13,01 € / ud

Si aceptas, añadimos esas 7 unidades a tu pedido: recibirás 8 en lugar de 1 y pagarás 80,97 € más (11,57 € por unidad en vez de 23,14 €). Tu pedido pasa de 23,14 € a 104,11 €.

Te salen a 13,01 € por unidad en lugar de 23,14 €.

Las unidades extra se fabrican expresamente para este pedido y no admiten devolución por desistimiento. Si no aceptas, recibirás las 1 unidades que pediste.

Al aumentar las unidades puede variar el coste de envío.

☐ Sí, quiero las 7 unidades extra por 80,97 € más. He leído las condiciones.

Si no marcas la casilla, recibirás solo las unidades que has pedido.

Este producto no suma puntos por estar rebajado.

Security policy

(edit with the Customer Reassurance module)

Delivery policy

(edit with the Customer Reassurance module)

Return policy

(edit with the Customer Reassurance module)

Descripción

Detalles del producto

Symbol of lightness and delicacy, the hummingbird evokes curiosity and joy. Studio Design' PolyFaune collection features classic products with colorful patterns, inspired by the traditional japanese origamis. To wear with a chino or jeans. The sublimation textile printing process provides an exceptional color rendering and a color, guaranteed overtime.

Comentarios (0)

No hay reseñas de clientes en este momento.

Ecom Experts

Pagina 15 de 21

Infórmese de nuestras últimas noticias y ofertas especiales

Su dirección de correo electrónico [SUSCRIBIRSE](#)

Puede darse de baja en cualquier momento. Para ello, consulte nuestra información de contacto en el aviso legal.

PRODUCTS Ofertas Novedades Los más vendidos	OUR COMPANY Envío Aviso legal Términos y condiciones Sobre nosotros Pago seguro Contacte con nosotros Mapa del sitio Tiendas	SU CUENTA Seguimiento del pedido Iniciar sesión Crear una cuenta Mis alertas	INFORMACIÓN DE LA TIENDA Tienda de pruebas España Envíenos un mensaje de correo electrónico: gusman126@gmail.com
---	---	---	---

[Cambiar Consentimiento de Cookies](#) [Desistir del contrato aquí](#)

© 2026 - Software Ecommerce desarrollado por PrestaShop™

El bloque se coloca justo encima del selector de cantidad y del botón *Añadir al carrito*, que es donde el cliente ya está mirando cuando decide comprar.

Personaliza tu producto

Texto grabado (+4,24 €)

Para Marta

Los campos marcados con * son obligatorios.

De arriba abajo: la cabecera configurada («Personaliza tu producto»), la etiqueta del campo con su sobreprecio ya con IVA («Texto grabado (+4,24 €)»), la caja donde escribe, y el pie con la letra pequeña. Si el campo tuviera límite de longitud, bajo la caja aparecería el contador de caracteres.

Al pulsar *Añadir al carrito*, el módulo valida antes de enviar: si falta un campo obligatorio o el texto no cumple la validación, sale un aviso y el producto no se añade. Ya dentro del carrito, cada línea muestra los datos que se escribieron, y dos líneas del mismo producto con textos distintos se tratan como líneas separadas (esa es la función del `values_hash`). El cliente vuelve a ver todo en **Historial de pedidos** → **Detalles**.

7. Preguntas frecuentes y problemas

«**El bloque no sale en la ficha del producto.**» Casi siempre es el selector de posición. Comprueba en *Settings* que **HTML tag for product fields** es un selector que existe en tu tema (en el *classic* de PrestaShop 8 es `.product-add-to-cart`). Si tu tema es propio, mira con el inspector del navegador qué contenedor envuelve al botón de añadir al carrito y pon ese. Lo segundo a mirar: que el grupo del cliente esté marcado en el campo.

«**El campo sale, con su precio, pero el carrito no cobra ese suplemento.**» Falta el override del carrito. Entra en la pantalla del módulo: si el aviso rojo del override está ahí, te dice exactamente por qué. O bien la carpeta `override/classes/` no tiene permisos de escritura, o bien ya hay un `Cart.php` de otro módulo y el

nuestro no lo pisa a propósito. En ese segundo caso hay que **combinar los dos ficheros a mano**: es trabajo de desarrollo, no se resuelve desde el back-office.

«**He cambiado algo y no se ve.**» Vacía la caché: *Settings* → **Clear Prestashop Cache** → *Save*. Si el cambio es de plantilla, además desactiva la caché de Smarty en *Parámetros avanzados* → *Rendimiento* mientras pruebas.

«**El contador de caracteres pone un número raro.**» Estaba pasando cuando el campo no tenía límite de longitud. Corregido en esta revisión: ahora, sin límite, el contador simplemente no se dibuja.

«**He cambiado de tema y el módulo dejó de pintar.**» *Soft reset module* vuelve a registrar todos los hooks sin tocar los datos. Anota antes la cabecera y el pie: esos dos vuelven a su valor de fábrica.

«**¿Puedo tener instalados este módulo y el `an_productfields` original a la vez?**» Sí, y es la razón de que los overrides se copien a mano. Comparten tablas y configuración, así que los campos que crees en uno los ve el otro. Lo que **no** debes hacer es tenerlos los dos activos: el bloque se pintaría dos veces en la ficha.

«**¿Se pierden los datos si desinstalo?**» No. La desinstalación normal deja las siete tablas intactas.

8. Seguridad

Se han revisado el fichero principal, el controlador de administración, el controlador de front, las dos clases, el override del carrito, las 24 plantillas y los cuatro ficheros JavaScript. Esto es lo que se ha encontrado y **corregido en esta revisión**:

Corregido

1. Subida de ficheros sin ninguna comprobación — el fallo grave

(`controllers/admin/AdminAnProductFields82Controller.php` , `ajaxProcessAddValue()`). La imagen que acompaña a un valor de lista se guardaba con `move_uploaded_file($_FILES['file']['tmp_name'], 'views/img/' . time() . $_FILES['file']['name'])` sin mirar **la extensión, el tipo real ni el nombre**. Dos agujeros a la vez: un empleado con permiso sobre el catálogo podía subir un `.php` a una carpeta pública y ejecutarlo, y un nombre con `../` escribía fuera de la carpeta del módulo. Ahora hay un método `storeUploadedImage()` que exige `UPLOAD_ERR_OK` , `is_uploaded_file()` , un tamaño máximo de 2 MB, una **lista blanca** de extensiones (jpg, jpeg, png, gif, webp), el **tipo real** comprobado con `getimagesize()` , y **el nombre lo pone el módulo** (`anpf_<hora>_<aleatorio>.<ext>`): nada de lo que llegue del navegador acaba en disco. Además se ha añadido `views/img/.htaccess` que apaga el intérprete de PHP y deniega el acceso a cualquier fichero ejecutable de esa carpeta.

2. Cualquiera podía leer los datos de cualquier pedido (`controllers/front/opc.php` , `getOrderDetailsAction()`). El endpoint hacía `new Order((int)Tools::getValue('id_order'))` y devolvía el HTML con los campos rellenos, **sin comprobar de quién era el pedido**. Un visitante sin cuenta podía recorrer los `id_order` de uno en uno y leerse lo que habían escrito todos los clientes de la tienda. Ahora se exige que haya cliente en sesión y que `$order->id_customer` sea el suyo; si no, devuelve vacío.

3. Los endpoints por hash actuaban sobre carritos ajenos (mismo fichero, `removeByHashAction()` y `updateQtyByHashAction()`). Borraban filas o cambiaban cantidades a partir del `values_hash` recibido, sin

mirar a qué carrito pertenecía. Se ha añadido `hashBelongsToCurrentCart()`, que comprueba contra `an_productfields_cart.id_cart` antes de tocar nada. `updateQtyByHash` además solo acepta ya `up` o `down` como dirección.

4. **Invocación de métodos por nombre desde la URL** (mismo fichero, `initContent()`). Se montaba el nombre del método con lo que llegara en `action` y se llamaba con `method_exists()`: quedaba expuesto cualquier método del controlador —propio o heredado— cuyo nombre acabara en `Action`. Sustituido por una **lista blanca** de las cinco acciones que el módulo usa de verdad.
5. **XSS almacenado en el back-office** (`views/templates/admin/fields_values.tpl`). El valor, el precio, el nombre del enlace, el idioma y la URL del fichero de cada valor de lista se imprimían **sin escapar**. Ahora llevan `|escape:'html':'UTF-8'`, y el identificador que va dentro del `onclick`, `|intval`. Lo mismo con la URL del fichero en el bloque del front (`views/templates/hook/product_buttons.tpl`, línea 105).
6. **Enlaces salientes por HTTP** (`views/templates/admin/fields.tpl`). Los tres enlaces del autor iban a `http://bit.ly/...`. Pasados a `https://` y con `rel="noopener noreferrer"` y `target="_blank"`.
7. **Avisos de PHP 8.2 en cada petición** (`an_productfields82.php`). Seis propiedades (`new`, `morenew`, `displayProduct`, `firsttime`, `reduction`, `taxCalculator`) se creaban al vuelo en el constructor y PHP 8.2 las marca como *deprecated*. Declaradas explícitamente.
8. **Contador de caracteres con un número imposible** (`views/js/front/front.js`). Un campo sin límite de longitud no lleva `maxlength` en el HTML, y el DOM devuelve `-1` —no `undefined`—, así que el guardián no lo cazaba y el cliente veía «(0/-1)» bajo la caja de texto. Ahora, sin límite, el contador no se pinta. De paso se ha cambiado `innerHTML` por `textContent`.

Lo que ya estaba bien

- **SQL**: todas las consultas del módulo pasan los datos del usuario por `(int)` o por `pSQL()`. No se ha encontrado ni una concatenación cruda.
- Ningún `eval`, `exec`, `shell_exec`, `system` ni `unserialize` de datos externos.
- Ningún `nofilter` en las plantillas.
- Ninguna clave de API ni contraseña escrita en el código; el módulo no llama a ningún servicio externo, así que no hay nada que verificar en TLS.
- Las cabeceras **anti-caché** del endpoint AJAX ya son las tres correctas y —bien visto— el fichero lleva un comentario explicando por qué **no** se manda `Content-Type: application/json`: con esa cabecera jQuery devuelve la respuesta ya convertida en objeto y el `JSON.parse()` del propio módulo revienta.
- `index.php` presente en **todas** las carpetas, incluida `views/img/`.
- Ningún bucle sobre datos históricos que dispare correos ni avisos a terceros: el módulo no manda nada por su cuenta.
- Los `foreach` sobre resultados de base de datos ya van protegidos con `is_array($x) ? $x : array()`.

Pendiente (no se ha tocado a propósito)

- **El override de `Cart.php` sigue siendo un override**. Es la pieza que suma el precio de los campos al carrito, y sustituirla por hooks es un rediseño completo del módulo, no un arreglo. El módulo ya hace lo

único razonable: instalarlo a mano, marcarlo, no pisar el de otro módulo, y avisar en pantalla cuando no ha podido ponerlo.

- **Las opciones «Product added to cart modal» reescriben una plantilla del tema `classic`**. Está avisado en la propia pantalla y guarda una copia del original, pero sigue siendo una modificación fuera de la carpeta del módulo. Se ha dejado como estaba porque quitarla cambiaría el comportamiento que el cliente ya tiene configurado.
- **El sistema de traducciones es el clásico**. Migrarlo al nuevo (ficheros `.xlf` con `d='Modules....'`) obliga a tocar las 24 plantillas y no aporta nada a la seguridad, así que se ha dejado para una revisión aparte. Hoy la pantalla del back-office se ve en inglés.
- **Los enlaces de «Support» y «Another modules» son acortadores `bit.ly` del autor original**. Se han pasado a HTTPS, pero un acortador puede repuntarse a otro destino en cualquier momento. Si el módulo se va a entregar a un cliente, conviene sustituirlos por la URL real del soporte.

9. Ficha técnica

Estructura

```
an_productfields82/
  an_productfields82.php      clase principal, sin namespace
  includer.php               carga las clases con require_once
  config_es.xml
  classes/AnProductFields82.class.php      ObjectModel del campo
  classes/AnProductFieldsProduct82.class.php  relación campo <-> producto
  controllers/admin/AdminAnProductFields82Controller.php
  controllers/front/opc.php      endpoint AJAX del front
  override_v8/classes/Cart.php   override copiado a mano
  sql/install.php  sql/uninstall.php
  translations/es.php
  views/css/  views/js/  views/img/  views/templates/
  index.php en cada carpeta
```

Tablas (prefijo de la tienda; compartidas con `an_productfields`)

Tabla	Qué guarda
<code>an_productfields</code>	La definición del campo: código, tipo, precio, orden, validación, obligatorio, grupos, categorías, longitud máxima, impuesto propio.
<code>an_productfields_lang</code>	El nombre y el placeholder por idioma.
<code>an_productfields_values</code>	Los valores de las listas, con <code>id_lang</code> , precio, nombre de enlace y fichero.
<code>an_productfields_shop</code>	Qué campo se ve en qué tienda.
<code>an_productfields_product</code>	Qué campo se activa en qué producto y en qué orden.
<code>an_productfields_cart</code>	Lo que el cliente rellenó, por carrito, producto y combinación, con su <code>values_hash</code> .
<code>an_productfields_cart_values</code>	La cantidad asociada a cada <code>values_hash</code> .

Hooks registrados (16)

`displayOrderDetail` · `displayAdminOrderTabOrder` · `displayAdminOrderContentOrder` · `displayAdminOrderTabLink` · `displayAdminOrderTabContent` · `actionAdminControllerSetMedia` · `actionPaymentModuleProductVarTplAfter` · `displayAdminProductsExtra` · `displayProductButtons` · `actionProductUpdate` · `actionCartSave` · `displayHeader` · `displayAdminAfterHeader` · `displayCartExtraProductActions` · `actionCartUpdateQuantityBefore` · `actionBeforeCartUpdateQty` · `sendMailAlterTemplateVars`. (`displayPDFInvoice` solo se registra en tiendas de la rama 1.6.)

Controladores

- Admin: `AdminAnProductFields82` (pestaña bajo *Catálogo*). Acciones AJAX: `ajaxProcessGetValues`, `ajaxProcessAddValue`, `ajaxProcessDeleteValue`, `ajaxProcessDisplayAdminProductExtra`.
- Front: `an_productfields82opc`. Acciones admitidas, ahora por lista blanca: `validateProductfields`, `calculateFullprice`, `updateQtyByHash`, `getOrderDetails`, `removeByHash`.

Claves de configuración (`ps_configuration`, por tienda salvo la última)

`an_pf_separate` · `an_pf_include_tax` · `an_pf_include_specific` · `an_pf_header_text` (por idioma) · `an_pf_footer_text` (por idioma) · `an_pf_modal_type` · `an_pf_dynamic_price` · `an_pf_display_prices` · `an_pf_text_counter` · `an_pf_js_price` · `an_pf_js_position` · `an_pf_js_position_type` · `an_pf_customvalidator` · `an_pf_override_payment_module` · `an_pf82_override_warnings` (global, uso interno: los avisos del override).

Ficheros que el módulo escribe fuera de su carpeta

- `override/classes/Cart.php` (y su copia `.an_pf_backup`).
- `themes/classic/modules/ps_shoppingcart/modal.tpl` (y su copia `modal-original.tpl`), solo si se usa la opción *Product added to cart modal*.
- `modules/an_productfields82/views/img/` para las imágenes de los valores de lista.

Cron, permisos y registros

El módulo **no tiene tareas programadas** ni escribe registros propios: sus únicos mensajes van a `PrestaShopLogger` cuando `addnewfeatures()` falla. La carpeta que necesita permisos de escritura es `override/classes/` (para el override) y `modules/an_productfields82/views/img/` (para las imágenes).

Migraciones

Todo lo que hay que añadir a una instalación anterior está en `addnewfeatures()`, dentro de un `try`, y se ejecuta al instalar y cada vez que se abre la pantalla del módulo: comprueba en `information_schema` las columnas que fueron apareciendo en versiones sucesivas, da de alta los hooks que falten y repone el override. No hay carpeta `upgrade/`.

Revisión y manual: septiembre de 2026. Instalado, configurado y capturado sobre PrestaShop 8.2.7 con datos de demostración.